

Colossal-Auto: 大规模模型并行化与激活检查点的统一自动化

Yuliang Liu^a, Shenggui Li^a, Jiarui Fang^a, Yanjun Shao^{a,b}, Boyuan Yao^{a,b}, Yang You^{*,c}

HPC-AI Technology Inc.^a 复旦大学^b 新加坡国立大学^c

* 通讯作者: youy@comp.nus.edu.sg

Abstract 近年来,大规模基础模型在多个领域持续刷新性能上限,但其训练过程也越来越依赖复杂的分布式并行与显存优化技术。流水并行、张量并行和激活检查点已经是常见手段;然而,现有工作大多分别优化分布式执行计划和激活检查点计划,缺少统一建模与联合搜索的方法。与此同时,静态规划又高度依赖精确的计算量与显存开销估计,而传统做法通常需要真实执行算子,代价高且容易失真。

本文提出 Colossal-Auto, 一个面向 PyTorch 的自动并行系统,用于联合搜索算子内并行 (intra-op parallelism) 与激活检查点策略。系统通过符号执行收集计算图与硬件信息,使用分层两阶段求解器在给定显存预算下搜索近似最优执行方案,并最终将普通的串行 PyTorch 模型重写为可直接运行的分布式版本。我们的实现还提供一个轻量级符号分析器,能够以极低成本为任意 PyTorch 模型估计内存与计算开销。源码已在 ColossalAI 中公开。

1 引言

基础模型 [4] 已成为人工智能领域的主流范式。BERT [7]、GPT-3 [5] 和 Vision Transformer [8] 等模型拥有海量参数,并在超大规模数据集上训练。随着参数量持续增长,模型性能通常随之提升;但代价是单卡 GPU 的高速显存越来越难以容纳训练过程中的参数、优化器状态与激活。即便 batch size 只有 1, 训练一个百亿参数模型也可能需要超过 80GB 的显存。因此,现代大模型训练通常必须采用分布式方案, GShard [18]、FairScale [1]、Megatron-LM [32] 和 DeepSpeed [28] 等系统都提供了不同的扩展策略。

然而,要为一个具体模型和具体集群配置找到最优分布式执行计划,仍然需要大量专家经验。以 SPMD 风格的算子内并行为例,搜索空间极其庞大:一方面,张量的不同维度都可能被切分;另一方面,设备又可以组织成任意维度的 device mesh。更复杂的是,一个算子的输出会成为下游算子的输入,而上下游对张量布局的要求可能完全不同,因此需要额外的 layout redistribution。已有方法通常只支持较低维度的 mesh 和切分方式,难以穷尽所有转换情形。与此同时,激活重计算 [6] 和模型数据卸载 [9, 30] 等显存优化策略又会显著改变可行搜索空间,使最优并行方案更加依赖全局联合考虑。

现有自动并行系统如 Alpa [37] 和 FlexFlow [15] 已展示出自动搜索执行计划的潜力,但仍存在两个明显限制。首先,激活检查点已经成为训练基础模型时几乎不可或缺的技术,而这些系统并没有把它作为联合优化对象。其次,已有方案在复杂硬件拓扑上的泛化能力不足,例如常见实现通常只把设备组织为二维 mesh,难以适配更复杂的连接结构。

本文提出一个基于 PyTorch [24] 的自动算子内并行系统,它能够把普通串行模型代码编译为高效的并行执行代码。除并行化外,系统还能在满足显存预算的前提下自动生成高效的激活检查点策略。为此,我们在分析阶段同时收集硬件信息和计算图信息,采用层次化优化方法搜索执行计划,并生成最终运行时代码。本文的主要贡献如下:

- 提出一种提前联合优化算子内并行与激活检查点的方法。
- 设计一种启发式张量布局转换算法,支持任意维度

device mesh 上不同 sharding spec 间的高效转换。

- 实现一个基于 meta-execution 的符号分析器,能够在细粒度计算图上同时收集计算与显存开销。

2 背景

由于 Nvidia GPU 等常见加速器的显存有限,单设备训练大模型很容易触发 OOM,因此分布式训练已成为突破单卡显存上限的必要手段。工业界和学术界已经提出了大量用于提升集群训练效率的方法,下面分别介绍与本文最相关的几类技术。

2.1 数据并行

数据并行是最常见的分布式训练方式,因为它实现简单且易于接入。在 N 张 GPU 上进行数据并行时,训练数据被切分为 N 份,模型副本复制到每张卡上,各卡用本地数据独立前向与反向,再在反向阶段同步梯度。Horovod [31] 和 PyTorch DistributedDataParallel [19] 都可以在较小代码改动下提供这种能力。

数据并行的主要问题是显存冗余:每张卡都要持有完整参数。随着 GPU 数量增加,吞吐量可以上升,但单卡可训练模型规模并不会自然扩大。为解决这一问题,DeepSpeed [28] 提出了 ZeRO [26],在数据并行过程中切分优化器状态、梯度和模型参数,从而显著降低冗余。此外,DeepSpeed 还支持 offloading [27],可将暂时不用的张量从 GPU 挪到 CPU 甚至 NVMe,进一步扩大可用内存空间。

2.2 流水并行

除按样本切分外,还可以按层切分模型。GPipe [13]、PipeDream [23]、Chimera [20] 和 Megatron-LM [32] 都采用了流水并行思路:将模型拆成若干连续层块,每个块映射到一个设备上,前向时传递激活,反向时传递梯度。这样模型规模会随设备数量增长而扩展。其代价是流水线气泡时间,即部分设备在某一时刻处于空闲。为减少

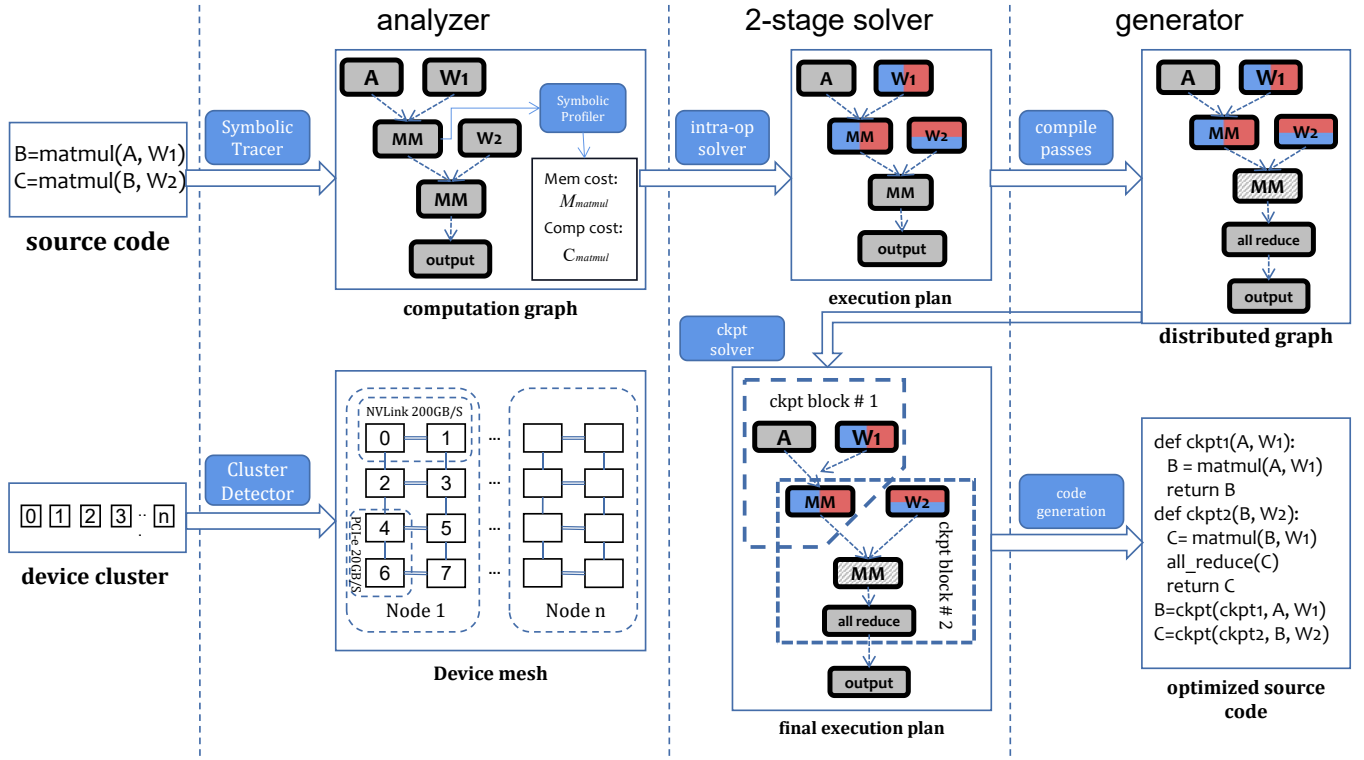


Figure 1: Colossal-Auto 的整体工作流

这一浪费，Megatron-LM 提出了交错流水，Chimera 提出了双向流水，都能显著降低 bubble time。

2.3 张量并行

张量并行通过切分参数张量并以 SPMD 方式执行训练。GShard [18] 允许用户为选定张量显式标注切分方案，并通过数据流分析推断其他张量的切分。Megatron-LM [32] 则面向 Transformer [33] 设计了行切分和列切分的线性层并行方式，应用于注意力和前馈模块。这样可以同时降低单卡计算量和显存占用，使模型扩展到数十亿参数。

张量并行与数据并行、流水并行相互正交，通常会组合使用。考虑到节点内通信通常快于跨节点通信，实践中张量并行往往局限在单机内，而跨机更多使用流水并行，以更好利用硬件带宽 [32]。

2.4 自动并行

上述策略大多需要用户显式制定规则，而自动并行系统则试图根据模型和硬件自动搜索执行方案。FlexFlow [22]、Tofu [34] 和 Alpa [37] 都属于这一方向。尤其是 Alpa，从新的视角统一了并行搜索：它把数据并行和张量并行都看作算子内并行，把流水并行视作算子间并行；前者用 ILP 搜索张量切分，后者借助 TeraPipe [21] 用动态规划搜索模块分段。

2.5 激活检查点

激活检查点 [6, 16] 通过以算换存的方式减少单卡显存占用。当对一组连续层启用检查点时，前向阶段只保存最后一层输出，而不保存中间激活；反向阶段再触发重计算，临时恢复中间结果用于梯度求解。这样可以显著降低中间激活的显存成本，为更大模型腾出空间。由于它本身不切分张量，所以与其他并行方式是正交的。Rotor [11]、Pofu [2]、Checkmate [14] 和 POET [25] 都研究了在给定显存预算下自动寻找最优检查点策略的问题。

3 设计

虽然前述技术已经显著降低了分布式训练门槛，但仍缺少一个能够把它们统一起来、真正做到自动化的系统。Alpa 已统一数据、张量和流水并行，但并未纳入激活检查点。本文希望把激活检查点也纳入统一规划之中，生成同时考虑并行方式与重计算策略的近似最优执行计划。当前实现主要聚焦于算子内并行；算子间并行会在后续工作中补充。

Colossal-Auto 建立在 PyTorch FX [29] 之上。系统输入是普通的、非分布式的 PyTorch 模型；输出则是已经应用高效算子内并行和自动选择激活检查点的 PyTorch 模型。由于输出仍然是 PyTorch 模型，它与 ZeRO-Offload [30]、PatrickStar [9, 12] 等运行时优化仍然兼容。示例如下。

```
model = resnet50(num_classes=10)
# convert original model to optimized model
```

```

model = autoparallelize(model, input_sample)

criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters())

# a normal training loop
for epoch in range(NUM_EPOCHS):
    for img, label in data_loader:
        optimizer.zero_grad()
        output = model(img)
        train_loss = criterion(output, label)
        train_loss.backward(train_loss)
        optimizer.step()

```

Listing 1: 用户只需极小改动即可接入系统的示例。

系统整体流程如图 1 所示，可分为三个核心部分：分析器、两阶段求解器和代码生成器。分析器负责提前收集计算图、算子开销和硬件拓扑信息；求解器在这些信息上搜索满足显存约束的高效执行计划；生成器再把计划应用回原始图，并重新编译出可运行代码。

4 分析器

分析器是为 PyTorch 构建的静态分析系统。由于 PyTorch 采用动态图执行，程序运行前很难直接获得完整图结构与精确开销信息。我们基于 PyTorch FX 模块 [29] 提前提取静态计算图，并在其上进行元信息分析。

分析器包含三个部分：用于收集计算与显存开销的符号分析器、用于探测硬件拓扑和带宽的集群探测器，以及用于搜索张量布局转换路径并记录转换代价的张量布局管理器。

4.1 符号分析器

为了评估并行策略和激活检查点策略，我们需要知道每个节点的浮点运算量与显存消耗。但在 PyTorch 的动态图语义下，这类信息通常只能在真实运行时获得，难以在搜索前离线使用。

PyTorch FX 提供的 `symbolic tracing` 使这一目标变得可行。符号跟踪不会真正执行算子，而只推断输出张量的元数据，例如 `shape` 和 `dtype`，如图 2 所示。我们在此基础上扩展了更广泛的操作、控制流以及 PyTorch meta tensor 支持。在跟踪过程中，系统为每个节点附加元信息，并通过符号执行收集其计算开销（TFLOPs）和显存开销（Byte）。由于整个过程只做模拟而不做真实计算，分析开销很低。

为评估分析器精度，我们在十余种主流模型上进行了实验，包括 VGG_16、ResNet50、ViT 和 GPT2 等。图 3 表明，符号分析得到的显存估计结果与真实执行值非常接近。

4.2 集群探测器

我们采用 Alpha [37] 中的 `device mesh` 概念来抽象参与训练的设备。`device mesh` 可以把一组物理设备组织成逻辑二维网格，例如 8 张 GPU 可以视为 1×8 或 2×4 mesh。在逻辑 mesh 中，同一维上的所有设备都参与相同类型的

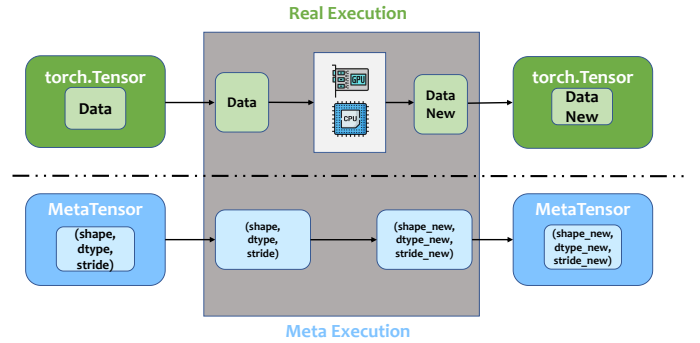


Figure 2: 符号执行示意

计算和集合通信，因此如果同一轴上的设备带宽差异较大，通信瓶颈就会由最慢链路决定。

Alpha 默认假设同一节点内设备间带宽一致，但现实集群常常更复杂，例如只有部分相邻 GPU 之间有 NVLink，其余只能走 PCI-e。为更好适配这类环境，我们的集群探测器会主动测量通信时延和带宽，并据此恢复更细粒度的拓扑结构。测得的带宽与时延信息不仅可用于构造更合理的 `device mesh`，也会进一步注入到后续代价模型中。

4.3 张量布局管理器

在算子内并行中，同一个张量可能有多种不同布局，因此需要一种表示方法来描述切分方式。我们沿用了 Alpha 的 SPMD 风格 `sharding spec`。对于一个 N 维张量，其 `sharding spec` 记为 $X_0 X_1 \dots X_{n-1}$ ，其中每个 X_i 属于 $\{S, R\}$ ；若某一维记为 S_j ，表示该维沿 `device mesh` 的第 j 个轴切分。若一个 S 带有多个下标，则表示沿多个 `mesh` 轴共同切分。

当上游和下游算子需要的 `sharding spec` 不一致时，就必须执行 `layout conversion`，也称 `redistribution`。现有方法大致有两类。枚举式方法预先穷举所有可能情况并查表转换，但随着 `mesh` 维度提升，枚举规模会迅速膨胀。逐维转换方法则按照张量维度顺序依次把源 `spec` 转成目标 `spec`，虽然总能得到一条可行路径，但通常并不高效。

为此，我们提出一种基于启发式搜索的转换算法，如算法 1 所示。算法从源 `spec` 出发，生成所有单步可达 `spec`，然后利用相似度函数选出与目标 `spec` “差异最小”的候选，作为下一步搜索起点，直到到达目标 `spec`。

单步变换。在我们的布局管理器中，会改变 `sharding spec` 的基础操作只有三种：`all-gather`、`shard` 和 `all-to-all`。对每个操作产生的单步合法转换都加入候选集合。例如， $S_0 R$ 的单步转换集合可以是 $[RR, S_0 S_1, S_{01} R, RS_0]$ 。如果某一转换要求张量维度大小不能整除对应 `mesh` 轴大小，则该转换无效。

启发式函数。我们按维度衡量两个 `sharding spec` 的差异，因此源 `spec` s 与目标 `spec` t 的差异可写为

$$\sum_{i \in (0, 1, \dots, \dim(s)-1)} \dim_diff(s[i], t[i])。$$

Dim difference 会累积每个维度所需转换操作的代价。这里的代价并不直接等于真实通信时间，而是用于描述

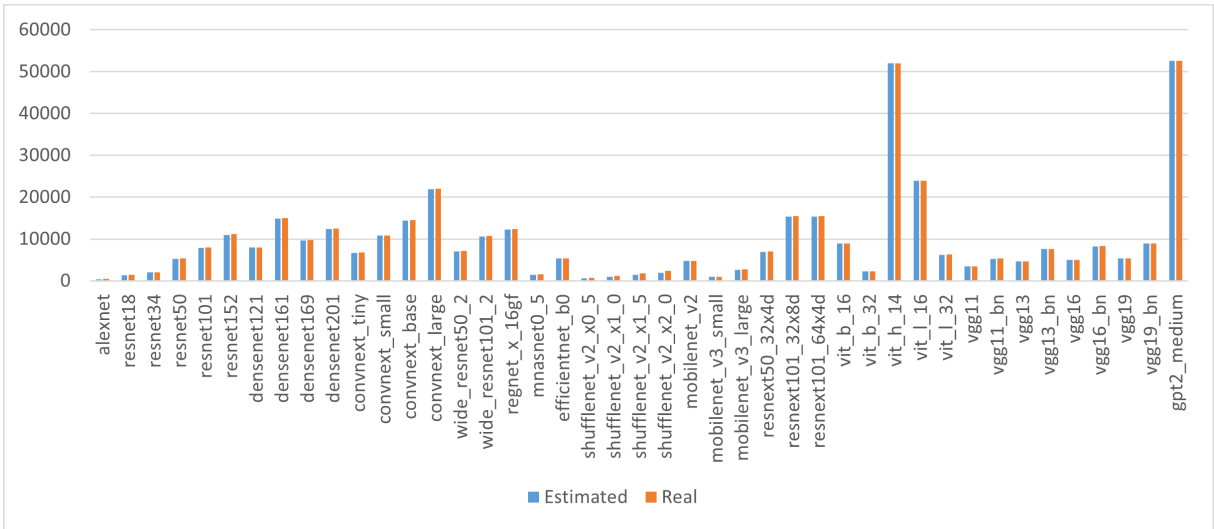


Figure 3: 符号显存分析精度评估

Algorithm 1: 张量布局转换路径搜索

```

Data: source_spec, target_spec
Result: transform_path
total_step  $\leftarrow 0$ ;
transform_path  $\leftarrow []$ ;
transform_path[total_step]  $\leftarrow source\_spec$ ;
while min_diff  $\neq 0$  do
    min_diff  $\leftarrow \infty$ ;
    valid_transforms  $\leftarrow$ 
        Transform(source_spec);
    forall sharding_spec  $\in$  valid_transforms do
        diff  $\leftarrow$ 
            Heuristic(sharding_spec, target_spec);
        if diff  $<$  min_diff then
            source_spec  $\leftarrow sharding\_spec$ ;
            transform_path(total_step)  $\leftarrow$ 
                source_spec;
        end
    end
    total_step  $+= 1$ ;
    transform_path(total_step)  $\leftarrow source\_spec$ ;
end
return transform_path;

```

两个 dim spec 之间的结构差异。由于 shard 是片上操作，而 all-gather 需要跨设备通信，我们给 all-gather 更高权重；若两个 dim spec 间需要多步转换，还会额外施加 step penalty。例如， $S_0 \rightarrow S_1$ 需要经过 $S_0 \rightarrow R \rightarrow S_1$ ，因此 $dim_diff(S_0, S_1) = cost(all_gather) + cost(shard) + step_penalty$ 。

对求解器的支持。布局管理器不仅给出转换路径，也会记录从源 spec 到目标 spec 的通信开销。该开销用 alpha-beta 代价模型估计，其中 alpha 和 beta 由集群探测器测得。为了避免运行时重复搜索，我们还已把已经求解过的源目标 spec 对缓存下来，运行时直接复用。

5 两阶段求解器

系统通过两个层次搜索执行计划：算子内并行和激活检查点。这样的层次化设计有两个直接好处。第一，算子内并行搜索时间与图中节点数量正相关，而未插入激活重计算的图更小，因此能显著降低求解时间。第二，以往激活检查点工作通常不考虑分布式训练中的通信开销，导致在分布式模型上性能欠佳；我们把通信成本显式纳入激活检查点建模，从而得到更好的联合方案。

5.1 算子内并行求解器

我们的求解器改造自 Alpha 的 ILP 算子内并行求解器 [37]，并加入若干工程优化，以兼顾通用性和求解效率。

由于 PyTorch FX 捕获的操作还不够底层，节点种类很多，若逐个节点为其匹配切分策略会导致复杂度很高。为此，我们增加了一层策略生成抽象：先使用 node dispatcher 根据操作类型把节点路由到对应的 strategy generator。例如，addbmm、bmm 和 matmul 可以共享同一个生成器，大多数一元逐元素操作也可以共享一类生成器。采用这一方式后，要支持 GPT2 模型仅需二十种以内的策略生成器。

我们还进一步做了几项复杂度削减。首先，借助 AoT 符号分析和分层搜索，算子内并行阶段只考虑前向图，从而显著减少节点数。其次，对 getitem、简单 elementwise、slice 等计算代价很低的节点进行合并。最后，那些既不接收张量输入也不产生张量输出的节点，例如标量加法，会直接从图中移除。通过这些优化，求解器能够在可接受时间内完成搜索。

5.2 激活检查点求解器

激活检查点求解器接收已经分布式的计算图，并搜索最优子图来应用 activation checkpoint。

5.2.1 建模

我们继承 Rotor [11] 的自动激活检查点算法。该算法把模型视为由 L 个阶段构成的链，每个阶段既可以是一层，也可以是一组层。对任意阶段 ℓ ，其前向激活 a^ℓ 和反向激活 $\delta^{\ell-1} = \frac{\partial \mathcal{L}}{\partial a^\ell}$ 可以写成：

$$\begin{aligned} F^\ell : a^\ell &= f_\ell(\theta^\ell, a^{\ell-1}) \\ B^\ell : \delta^{\ell-1} &= \bar{f}_\ell(\theta^\ell, \delta^\ell, \bar{a}^\ell, a^{\ell-1}) \\ \frac{\partial \mathcal{L}}{\partial \theta^\ell} &= \bar{g}_\ell(\delta^\ell, \bar{a}^\ell, a^{\ell-1}) \end{aligned} \quad (1)$$

其中， θ^ℓ 表示阶段 ℓ 前向计算涉及的参数集合， $\bar{a}^\ell$ 表示反向传播所需的中间激活集合。

在此基础上，激活检查点调度涉及的主要操作列于表 1。

操作	输入	输出	时间	显存开销
F_{all}^ℓ	$\{a^{\ell-1}\}$ $\{\bar{a}^{\ell-1}\}$	$\{a^{\ell-1}, \bar{a}^\ell\}$ $\{\bar{a}^{\ell-1}, \bar{a}^\ell\}$	u_f^ℓ	o_f^ℓ
F_{ck}^ℓ	$\{a^{\ell-1}\}$ $\{\bar{a}^{\ell-1}\}$	$\{a^{\ell-1}, a^\ell\}$ $\{\bar{a}^{\ell-1}, a^\ell\}$	u_f^ℓ	o_f^ℓ
$F_\emptyset^\ell$	$\{a^{\ell-1}\}$	$\{a^\ell\}$	u_f^ℓ	o_f^ℓ
B^ℓ	$\{\delta^\ell, \bar{a}^\ell, a^{\ell-1}\}$ $\{\delta^\ell, \bar{a}^\ell, \bar{a}^{\ell-1}\}$	$\{\delta^{\ell-1}\}$ $\{\delta^{\ell-1}, \bar{a}^{\ell-1}\}$	u_b^ℓ	o_b^ℓ

Table 1: 激活检查点相关操作

为了与算子内并行求解器衔接，我们进一步把每个阶段上的通信时间和通信显存开销也纳入建模，记号如表 2。

阶段	时间	显存开销
前向通信	u_{fcomm}^ℓ	o_{fcomm}^ℓ
反向通信	u_{bcomm}^ℓ	o_{bcomm}^ℓ

Table 2: 通信开销

据此，我们把 Rotor 中的主定理扩展为考虑通信成本的版本：

定理 1. $C_{BP}(s, t, m)$ 表示在可用显存为 m 时，从阶段 s 到阶段 t ($t \geq s$) 执行任意合法 *persistent sequence* 的最优时间，则有

$$\begin{aligned} C_{BP}(s, s, m) &= \begin{cases} u_f^s + u_{fcomm}^s + u_b^s + u_{bcomm}^s & m \geq m_{all}^{s,s} \\ \infty & m < m_{all}^{s,s} \end{cases} \\ C_{BP}(s, t, m) &= \min(C_1(s, t, m), C_2(s, t, m)) \end{aligned} \quad (2)$$

其中

$$\begin{aligned} C_1(s, t, m) &= \begin{cases} \min_{s'=s+1 \dots t} C_{ck}(s, s', t, m) & m \geq m_\emptyset^{s,t} \\ \infty & m < m_\emptyset^{s,t} \end{cases} \\ C_2(s, t, m) &= \begin{cases} C_{all}(s, t, m) & m \geq m_{all}^{s,t} \\ \infty & m < m_{all}^{s,t} \end{cases} \end{aligned} \quad (3)$$

且

$$\begin{aligned} C_{ck}(s, s', t, m) &= \sum_{k=s}^{s'-1} u_f^k + C_{BP}(s', t, m - \omega_a^{s'-1}) \\ &\quad + C_{BP}(s, s' - 1, m) \\ C_{all}(s, t, m) &= u_f^s + u_{fcomm}^s + C_{BP}(s + 1, t, m - \omega_a^s) \\ &\quad + u_b^s + u_{bcomm}^s \end{aligned} \quad (4)$$

并且

$$\begin{aligned} m_\emptyset^{s,t} &= \max \left\{ \omega_\delta^t + \omega_a^s + o_f^s + o_{fcomm}^s, \omega_\delta^t + \max_{s+1 \leq j < t} \left\{ \omega_a^{j-1} + \omega_a^j + o_f^j + o_{fcomm}^j \right\} \right\} \\ m_{all}^{s,t} &= \max \left\{ \omega_\delta^t + \omega_a^s + o_f^s + o_{fcomm}^s, \omega_\delta^s + \omega_a^s + o_b^s + o_{bcomm}^s \right\} \end{aligned} \quad (5)$$

沿用 Rotor 的动态规划过程，只需把原公式替换为上述扩展版本，就可以在激活检查点搜索时自然纳入算子内并行引入的通信成本。

5.2.2 线性化

先前工作 [2, 11] 通常要求网络满足线性化假设，也就是模型应近似为串行结构：一层的输出只传给下一层，而不额外分支到其他层，如图 4 所示。

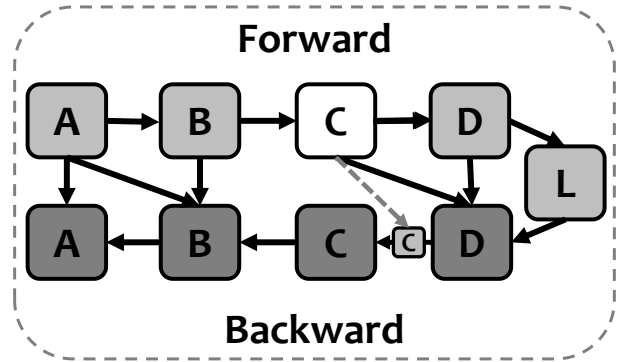


Figure 4: 线性化计算图

但现代模型通常并不满足这一假设，ResNet [10] 和 Transformer [33] 中都存在大量残差连接。过去的做法往往需要人工把模型重写为 `torch.nn.Sequential`，这对研究者和工程师都不够友好。借助 `torch.fx`，我们可以直接访问前向计算图及节点的父子关系，于是可以在图上做依赖分析，自动寻找合适的分段位置，使求解器可以在 `node group` 粒度上满足线性化要求。

定义 1. 我们在网络线性化中使用如下概念：

- **Node**: 计算图中的节点，对应一个操作。
- **Graph**: 神经网络的计算图，默认视为有向无环图 (DAG)。

- **Node group**: 求解器真正处理的“节点”；线性化后网络由一系列 *node group* 构成。
- **End of a node group**: 按拓扑序遍历图时，把节点持续追加到当前 *group* 中；若某节点被判定为当前 *group* 的结束点，则之后开始构造下一个 *group*。
- **Existing dependencies**: 动态维护的依赖池。处理某节点时，先移除其父节点对依赖池的贡献，再把它对子节点引入的新依赖加入池中。

依据上述定义，我们的基本规则是：若某节点在移除其父节点依赖后，依赖池中的现存依赖全部清零，则把该节点视为一个 *node group* 的结束点。这样就可以自动线性化诸如 ResNet-152 [10] 这类经典残差网络。

5.2.3 Common Node

尽管上述规则已能在线性化许多网络，但 Transformer 类模型仍带来额外挑战。这类模型经常在训练过程中使用 *attention mask* 等常量，而这些常量节点会被大量算子共享。如果不特殊处理，依赖关系会把整张图粘连成一个超大 *group*，无法有效线性化。

[17] 引入了 *common node* 机制来消除此类依赖，但其识别方式主要依赖“子节点数量超过阈值”的经验规则。我们给出一个更明确的定义：

定义 2. 若某节点是不可微的，则将其标记为 *common node*。例如 PyTorch 中的 *attention mask* 常为 *bool* 类型，不会在训练中创建梯度；类似 *getattr*、*getitem* 这类 Python 操作同样不可微。

进一步地，*common node* 的属性可以在图中传播：

引理 1. 若满足以下任一条件，则某节点也会被标记为 *common node*：

- 该节点的所有父节点都已被标记为 *common node*；
- 该节点对应的操作本身不可微，例如 *getattr*、*getitem* 等。

实际操作时，只需把网络输入中的原始 *common node*（例如 Transformer 的 *attention mask*）先标记出来，再按上述规则传播，就能扩展 *common node* 集合，并在网络线性化时忽略这些节点的依赖影响。

5.2.4 完整线性化规则

在上述机制之外，还有两点需要在线性化中同时考虑。第一，算子内并行会引入通信节点，但在激活检查点建模时无需把它们显式拆成额外阶段，只需把通信节点与其父节点放在同一个 *group* 中。第二，现代 DNN 中广泛存在 *in-place* 操作，例如 ResNet 里 BatchNorm 后的原地 ReLU。把这类操作及其父节点一并归组，有助于更准确地估计显存占用。

综合考虑后，我们得到最终的 *网络线性化算法*，如算法 2 所示。

Algorithm 2: 网络线性化算法

Data: 计算图 $\mathcal{G}$, *common node* 集合 $\mathcal{C}$
Result: 线性化后的图 $\bar{\mathcal{G}}$
Function *is_sink*(*deps_pool*, *Node*):
 if *deps_pool* 为空, 且 *Node.children* 中不存在 *in-place* 或通信操作 **then**
 return True;
 else
 return False;
 end
 $\bar{\mathcal{G}} \leftarrow []$;
 $Current_Node \leftarrow []$;
 $deps_pool \leftarrow \{\}$;
for *Node* **in** $\mathcal{G}$ **do**
 if *Node* 不是 *placeholder* **then**
 for *Parent* **in** *Node.parents* **do**
 if *Parent* 不是 *placeholder* 且属于 *common node* **then**
 $deps_pool[Parent] - = 1$;
 end
 end
 append *Node* to *Current_Node*;
 if *is_sink*(*deps_pool*, *Node*) **then**
 append *Current_Node* to $\bar{\mathcal{G}}$;
 $Current_Node \leftarrow []$;
 end
 if *Node* 满足引理 1 **then**
 append *Node* to $\mathcal{C}$;
 end
 $deps_pool[Node] \leftarrow len(Node.children)$;
 end
end
return $\bar{\mathcal{G}}$;

5.3 联合两个求解器

两个求解器虽然都以“在显存预算下最小化执行时间”为目标，但他们对显存预算的使用方式不同。如果二者共享同一个固定预算，算子内并行求解器通常已经会把显存压到预算边界，从而让激活检查点阶段失去作用；如果把两类决策完全联合建模，又会导致求解复杂度过高，难以在可接受时间内完成。

为此，我们让算子内并行求解器在一组离散预算 $[(1 + \alpha)^n]$ 上分别求解，其中 $n \in [0, 9]$ 、 α 为扩展系数。对于每个预算点，都得到一个对应的最优算子内并行方案，再将其输入激活检查点求解器。最终从所有预算点产生的候选计划中，选出总执行时间最短的那个作为最终执行计划。

6 生成器

生成器负责把原始的非分布式计算图转换为应用最优执行计划后的图，并将其重新编译成 PyTorch 代码。为实现这一点，我们设计了一组图编译 pass，以及面向优化后 Graph IR 的代码生成逻辑。

6.1 编译 pass

首先，我们设计了与通信相关的 pass，根据执行计划插入所需通信节点。这里主要有两类通信：一类是保证数值正确性的通信，例如局部求和后插入 all-reduce；另一类是为了把张量转换成下游节点要求的布局而执行的 redistribution。

其次，我们实现了参数切分 pass，用于对模块参数进行分片，并额外添加 hook 来处理梯度通信。为了提高训练效率，我们使用额外的 CUDA stream，让异步梯度通信尽可能与反向计算重叠。

最后，我们加入 reshape conversion pass，用来调整 transpose、reshape 等节点的参数，使其适配搜索得到的执行计划。这样做的原因是，某些 reshape 类操作的参数可能仍基于原始输入张量的形状，而不是切分后张量的形状。

6.2 代码生成

PyTorch FX [29] 原生支持从给定 Graph 生成语义等价的 Python 代码。为了支持自动激活检查点，我们对其代码生成流程做了扩展。

在激活检查点搜索结束后，我们会为每个节点打上所属 checkpoint block 的索引。所有索引相同的节点会被打包成一个 activation checkpoint block。代码生成器首先为每个 block 生成一个封装函数，然后再用 PyTorch 内置的 checkpoint 机制包裹这些函数，从而在反向阶段自动保留必要输入并触发重计算。最后，原始计算图中属于该 block 的节点会被替换为相应的 checkpoint 包装调用。

7 评估

实验在 8 张 Nvidia A100 (80GB) GPU 上进行。需要注意的是，其中只有 4 对相邻 GPU 之间通过 NVLink 互连，其余链路通过 PCI-e 连接。

实验 ID	GPU 数量	Hidden size	参数量 (十亿)
α	1	2048	0.409
β	2	4096	1.221
γ	4	8192	4.053
δ	8	16384	14.550

Table 3: 实验中 GPT2 的模型配置。层数固定为 4，序列长度固定为 1024。之所以选择相对较少的层数，是因为流水并行通常会把模型切成若干分区，而每个分区中的 Transformer block 数量往往与这里的设置处于同一量级。在 weak scaling 设置下，问题规模会随设备数量同步扩大。我们将本方案与 Megatron-LM [32] (1D tensor parallelism)、Optimus [35] (2D tensor parallelism) 以及 3D tensor parallelism [3] 进行比较。具体配置见表 3，性能结果见表 4。随着问题规模增大，DDP 会出现 OOM，因此未纳入比较。

实验 ID	Megatron	Optimus	3D TP	本文方法
α	0.161	0.161	0.161	0.161
β	0.324	-	-	0.332
γ	0.528	0.368	-	0.604
δ	0.728	-	0.715	0.824

Table 4: Weak scaling 性能。模型设置见表 3。这里用训练过程总 PFLQPS 衡量性能。结果表明，本文系统在所有设置下都取得了最佳性能。原因在于我们的求解器显式利用了集群拓扑信息，使执行计划能够自适应硬件结构。例如，在该实验环境中，系统会在跨 NUMA 节点的设备之间更倾向于采用类似数据并行的策略，而在同一 NUMA 节点内的 4 张 GPU 之间采用类似张量并行的策略。这样既避免了跨 NUMA PCI-e 链路上的高成本张量并行通信，又能把低带宽链路上的通信与反向计算重叠起来。

8 未来工作

8.1 为更多算子构建策略生成器

在算子内并行阶段，我们需要为每个算子生成全部合法并行策略。尽管当前只需不到二十类 strategy generator 就能支持 GPT2，但要覆盖 PyTorch 中约 2000 个算子仍然是一项繁重工作。随着 PyTorch 2.0 的 PrimTorch 把约 2000 个算子规范化到约 250 个原语算子上，这一问题会明显缓解，也更有可能真正枚举完整策略集合。

8.2 两阶段求解器的系统性评估

本文已经在 ResNet50 [10] 上应用了两阶段求解器，但针对基础模型 [4] 的更完整评估仍需后续补充。一些实验仍涉及工程性 patch，未来会进一步完善。

8.3 支持算子间并行

当前系统只联合搜索算子内并行和激活检查点。后续我们将把算子间并行也纳入统一执行计划，并沿用本文的自动化方法进行搜索。

9 结论

本文提出 Colossal-Auto，用于联合搜索算子内并行与激活检查点，从而为大规模模型生成近似最优的分布式训练方案。系统通过分析器收集硬件性能与计算图代价信息，利用两阶段求解器搜索高质量执行计划，并最终把普通 PyTorch 模型重新编译为分布式模型。高效的分布式训练策略设计一直高度依赖专家经验；我们希望该系统能够显著降低大模型训练的使用门槛，并在后续工作中继续提升其鲁棒性和通用性。

References

- [1] FairScale authors. Fairscale: A general purpose modular pytorch library for high performance and large scale training. <https://github.com/facebookresearch/fairscale>, 2021.
- [2] Olivier Beaumont, Lionel Eyraud-Dubois, and Alena Shilova. Efficient combination of rematerialization and offloading for training dnn. *Advances in Neural Information Processing Systems*, 34:23844–23857, 2021.
- [3] Zhengda Bian, Qifan Xu, Boxiang Wang, and Yang You. Maximizing parallelism in distributed training for huge neural networks, 2021.
- [4] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.
- [5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020. URL <https://arxiv.org/abs/2005.14165>.
- [6] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423>.
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2020. URL <https://arxiv.org/abs/2010.11929>.
- [9] Jiarui Fang, Zilin Zhu, Shenggui Li, Hui Su, Yang Yu, Jie Zhou, and Yang You. Parallel training of pre-trained

- models via chunk-based dynamic memory management. *IEEE Transactions on Parallel and Distributed Systems*, 34(1):304–315, 2022.
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [11] Julien Herrmann, Olivier Beaumont, Lionel Eyraud-Dubois, Julien Hermann, Alexis Joly, and Alena Shilova. Optimal checkpointing for heterogeneous chains: how to train deep neural networks with limited memory. *arXiv preprint arXiv:1911.13214*, 2019.
- [12] Haichen Huang, Jiarui Fang, Hongxin Liu, Shenggui Li, and Yang You. Elixir: Train a large language model on a small gpu cluster. *arXiv preprint arXiv:2212.05339*, 2022.
- [13] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, Hyoungho Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. *GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- [14] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Ghلامي, Pieter Abbeel, Joseph Gonzalez, Kurt Keutzer, and Ion Stoica. Checkmate: Breaking the memory wall with optimal tensor rematerialization. *Proceedings of Machine Learning and Systems*, 2:497–511, 2020.
- [15] Zhihao Jia, Matei Zaharia, and Alex Aiken. Beyond data and model parallelism for deep neural networks. *Proceedings of Machine Learning and Systems*, 1:1–13, 2019.
- [16] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. Dynamic tensor rematerialization. *CoRR*, abs/2006.09616, 2020. URL <https://arxiv.org/abs/2006.09616>.
- [17] Zhiqian Lai, Shengwei Li, Xudong Tang, Keshi Ge, Weijie Liu, Yabo Duan, Linbo Qiao, and Dongsheng Li. Merak: A efficient distributed dnn training framework with automated 3d parallelism for giant foundation models. *arXiv preprint arXiv:2206.04959*, 2022.
- [18] Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL <https://openreview.net/forum?id=qrwe7XHTmYb>.
- [19] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. Pytorch distributed: Experiences on accelerating data parallel training. *Proc. VLDB Endow.*, 13(12):3005–3018, sep 2020. ISSN 2150-8097. doi: 10.14778/3415478.3415530. URL <https://doi.org/10.14778/3415478.3415530>.
- [20] Shigang Li and Torsten Hoefler. Chimera: Efficiently training large-scale neural networks with bidirectional pipelines. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC ’21, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450384421. doi: 10.1145/3458817.3476145. URL <https://doi.org/10.1145/3458817.3476145>.
- [21] Zhuohan Li, Siyuan Zhuang, Shiyuan Guo, Danyang Zhuo, Hao Zhang, Dawn Song, and Ion Stoica. Terapipe: Token-level pipeline parallelism for training large-scale language models, 2021. URL <https://arxiv.org/abs/2102.07988>.
- [22] Wenyan Lu, Guihai Yan, Jiajun Li, Shijun Gong, Yinhe Han, and Xiaowei Li. Flexflow: A flexible dataflow accelerator architecture for convolutional neural networks. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 553–564, 2017. doi: 10.1109/HPCA.2017.29.
- [23] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. Pipedream: Generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP ’19, page 1–15, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450368735. doi: 10.1145/3341301.3359646. URL <https://doi.org/10.1145/3341301.3359646>.
- [24] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [25] Shishir G Patil, Paras Jain, Prabal Dutta, Ion Stoica, and Joseph Gonzalez. Poet: Training neural networks on tiny devices with integrated rematerialization and paging. In *International Conference on Machine Learning*, pages 17573–17583. PMLR, 2022.
- [26] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing*,

Networking, Storage and Analysis, pages 1–16. IEEE, 2020.

- [27] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning, 2021. URL <https://arxiv.org/abs/2104.07857>.
- [28] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '20*, page 3505–3506, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450379984. doi: 10.1145/3394486.3406703. URL <https://doi.org/10.1145/3394486.3406703>.
- [29] James K. Reed, Zachary DeVito, Horace He, Ansley Ussery, and Jason Ansel. Torch.fx: Practical program capture and transformation for deep learning in python, 2021. URL <https://arxiv.org/abs/2112.08429>.
- [30] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. Zero-offload: Democratizing billion-scale model training. In *USENIX Annual Technical Conference*, pages 551–564, 2021.
- [31] Alexander Sergeev and Mike Del Balso. Horovod: fast and easy distributed deep learning in TensorFlow. *arXiv preprint arXiv:1802.05799*, 2018.
- [32] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [33] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL <https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>.
- [34] Minjie Wang, Chien-chin Huang, and Jinyang Li. Supporting very large models using automatic dataflow graph partitioning. In *Proceedings of the Fourteenth EuroSys Conference 2019, EuroSys '19*, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450362818. doi: 10.1145/3302424.3303953. URL <https://doi.org/10.1145/3302424.3303953>.
- [35] Qifan Xu, Shenggui Li, Chaoyu Gong, and Yang You. An efficient 2d method for training super-large deep learning models, 2021.
- [36] Yuanzhong Xu, HyoukJoong Lee, Dehao Chen, Blake Hechtman, Yanping Huang, Rahul Joshi, Maxim Krikun, Dmitry Lepikhin, Andy Ly, Marcello Maggioni, et al. Gspmd: general and scalable parallelization for ml computation graphs. *arXiv preprint arXiv:2105.04663*, 2021.
- [37] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 559–578, 2022.